CS 162 Computer Architecture

Winter 2026

Course Project (100 pts)

Distributed: Jan 28th, 2026

Due: 11:59pm Mar 10th, 2026

Introduction:

This is a single-person project.

You are allowed and encouraged to discuss the project with your classmates, but **no sharing of the project source code and report**. Please list your discussion peers, if any, in your report submission.

One benefit of a dynamically scheduled processor is its ability to tolerate changes in latency or issue capability in out of order speculative processors.

The purpose of this project is to evaluate this effect of different architecture parameters on a CPU design by simulating a modified (and simplified) version of the PowerPc 604 and 620 architectures. We will assume a 32-bit architecture that executes a subset of the RISC V ISA which consists of the following 8 instructions: fld, fsd, add, addi, fadd, fmul, fdiv, bne. See Appendix A in the textbook for instructions' syntax and semantics.

Your simulator should take an input file as a command line input. This input file, for example, prog.dat, will contain a RISC V assembly language program (code segment). Each line in the input file is a RISC V instruction from the aforementioned eight instructions. Your simulator should read this input file, recognize the instructions, recognize the different fields of the instructions, and simulate their execution on the architecture described below in this handout. You will have to implement the **functional+timing** simulator.

Please read the following a-g carefully before you start constructing your simulator.

The simulated architecture is a **non-speculative, single-issue, out of order CPU using Tomasulo's Algorithm (without a Reorder Buffer)** where:

a. The fetch unit fetches NF=1 instruction every cycle.

b. No Branch Prediction is used. When a branch instruction (bne) is fetched/decoded, the Fetch unit stalls (stops fetching new instructions) until the branch instruction completes execution in the Branch Unit and the correct next PC is determined.

c. The decode unit can hold only NI=1 instruction (note: for the Bonus part of the project, you will be asked to increase this size). It acts as a simple pipeline latch between Fetch and Issue. If the Issue stage is stalled, the Decode stage holds the instruction and signals the Fetch unit to stall.

d. Up to NW=1 instruction can be issued every clock cycle from the instruction queue to a free reservation station. The architecture has the following functional units with the shown latencies and number of reservation stations:

Unit	**Latency (cycles) for operation**	**Reservation stations**	**Instructions executing on the unit**
INT	1 (integer and logic operations)	4	add, addi
Load/Store	2	3	fld, fsd
FPadd	3	3	fadd
FPmult	4	2	fmul
FPdiv	6	1	fdiv
BU	1 (condition and target evaluation)	1	bne

e. A single Common Data Bus (NB=1) is used to connect the functional units to the Register File and the Reservation Stations.

- No Reorder Buffer (ROB) is used. Instructions "commit" effectively as soon as they finish execution and write their result to the CDB.
- Contention Policy: Since there is only one CDB, if multiple functional units finish execution in the same cycle, the unit with the oldest instruction (based on issue order) gets priority to broadcast its result. The other units must stall and wait for the next cycle to retry broadcasting.

f. You need to implement implicit register renaming using the Reservation Station IDs as tags.

- No Physical Register File: You do not need to implement a separate physical register file or a free list.
- Register Status Table: Instead, use a Register Status Table (often called a RAT or Register Result Status). For each architectural register (R0-R31, F0-F31), this table tracks whether the register's value is currently valid in the register file, or if it is pending a result from a specific Reservation Station ID (e.g., "Waiting for RS#3").

g. Forwarding: You must implement "forwarding" via CDB. This means that when a result is broadcast on the CDB, any Reservation Station waiting for that specific tag must capture the value immediately in the same cycle, rather than waiting for the value to be written to the register file and read back later.

To simplify the simulation, we will assume that the entire program fits in the instruction cache (i.e., it always takes one cycle to read a cache line). Also, the data cache is perfect (hit rate = 100%) and single-ported. Reading or writing a word always takes one cycle. There are no cache misses to simulate. You may assume a logical separation between Instruction Memory and Data Memory. Use the addresses specified in the input file for data memory. Instructions are stored in a separate list starting at Index 0. The Program Counter (PC) simply tracks the index of the instruction in this list (PC, PC+1, PC+2...).

Your simulation should keep statistics about:

- **Total Execution Cycles:** The number of cycles from the start of the program until the last instruction completes.
- **Stall Events:** The number of times the instruction issue stage had to stall because the reservation stations of a given unit were full (Structural Hazard).

These statistics will help you identify the bottlenecks of the architecture (e.g., is the adder unit too slow? Do we need more reservation stations?).

Your simulation must be both **functionally correct** and **timing correct**.

- Functional Correctness: We will check the final contents of the Register File and Memory to ensure the program calculated the right answers.
- Timing Correctness: We will check the Total Execution Cycles to ensure your pipeline logic (hazards, stalls, and latencies) is implemented correctly.

Project language:

You can choose C/C++ or Python to implement your project.

Test benchmark

Use the following as an initial benchmark (i.e. content of the input file prog.dat).

%All the registers have the initial value of 0.
%memory content in the form of address, value.
0, 111
8, 14
16, 5
24, 10
100, 2
108, 27
116, 3
124, 8
200, 12

```
        addi    R1, R0, 24
        addi    R2, R0, 124
        fld     F2, 200(R0)
loop:   fld     F0, 0(R1)
        fmul    F0, F0, F2
        fld     F4, 0(R2)
        fadd    F0, F0, F4
        fsd     F0, 0(R2)
        addi    R1, R1, -8
        addi    R2, R2, -8
        bne     R1,$0, loop
```

(Note that this is just a testbench for you to verify your design. Your submission should support **ALL** the instructions listed in the table and you should verify and ensure the simulation correctness for different programs that use those eight instructions. When you submit your code, we will use more complicated programs to test your submission).

Project submission:

You submission will include two parts: i) code package and ii) project report

1. Code package:
   a. include all the source code files with **code comments**.
   b. have a README file 1) with the instructions to compile your source code and 2) with a description of your command line parameters/configurations and instructions of how to run your simulator.
2. Project report
   a. Describe the module design of your code, mark and list the key data structures used in your code.
   b. The results of above test benchmark.
   c. Your discussion peers and a brief summary of your discussion if any.

Project grading:

1. We will test the **timing** and **function** of your simulator using more complicated programs consisting of the eight RISC V instructions.
2. We will ask you later to setup a demo to test your code correctness in a 1-on-1 fashion.
3. We will check your code design and credits are given to code structure, module design, and code comments.
4. We will check your report for the design details.
5. Refer to syllabus for Academic Integrity violation penalties

Bonus (10 pts):

The simulator is implemented in a parameterized way so that one can experiment with different values of:

- NI: The size of the Instruction Queue (Decode Buffer).
- RS_SIZE: The number of Reservation Stations available for the different functional units (e.g., testing with 2 vs 16 stations).

Comparative analysis:

After running the benchmarks with the default parameters, perform the following experiments and provide the results/analysis in your report:

1. Effect of Instruction Queue Size: Study the effect of restricting the pre-decode buffer. Run the simulation with NI = 4 and 16. Does a smaller buffer cause more stalls in the Fetch unit?
2. Effect of Reservation Station Count: Study the effect of limiting the available instruction windows. Set the number of reservation stations for ALL functional units to 2. Compare the performance (execution cycles) against the default configuration.

You need to provide the results and analysis in your project report.